

쿠버네티스 환경에서의 PaC 위반 대응 및 예외 관리 시스템 설계



부산대학교 정보컴퓨터공학부

팀명 : 팩

C - 네트워크/통신 분과

지도교수 : 유영환

yeongrimgo1106@pusan.ac.kr 202355702 고영림

pty4pp@pusan.ac.kr 202155559 박태영

yuyeol@pusan.ac.kr 202148124 최유열

목차

1. 목표
2. 요구조건 및 제약사항 분석에 대한 수정사항
 - 2.1 요구조건 수정사항
 - 2.2 제약 사항 분석 수정사항
3. 설계 상세화 및 변경 내역
 - 3.1 시스템 구성 상세화
 - 3.2 인증·인가 구현 결과
 - 3.3 핵심 업무 흐름
 - 3.4 변경 내역
4. 구성원별 진척도
 - 4.1 갱신된 과제 추진 계획
 - 4.2 구성원별 수행 내용
5. 과제 수행 내용 및 중간 결과
 - 5.1 수행 내용
 - 5.2 중간 결과
 - 5.3 현재 한계
 - 5.4 향후 추진 계획

1. 목표

최근 클라우드 네이티브 환경에서는 Kubernetes 기반 애플리케이션 배포가 보편화되고 있다. Kubernetes는 컨테이너 기반 애플리케이션을 효율적으로 배포하고 운영할 수 있는 환경을 제공하지만, 잘못된 리소스 설정이나 보안 기준 위반이 그대로 배포될 경우 운영 장애나 보안 취약점으로 이어질 수 있다.

이러한 문제를 방지하기 위해 Policy-as-Code(이하 PaC) 방식의 정책 엔진을 활용하는 사례가 증가하고 있다. PaC는 보안 정책이나 운영 정책을 코드 형태로 정의하고, 배포 과정 또는 Kubernetes admission 단계에서 자동으로 검증하는 방식이다. 일반적인 단일 노드 운영체제가 하드웨어 리소스를 추상화하고 프로세스를 제어하는 것처럼, Kubernetes는 분산된 컴퓨팅 자원을 소프트웨어적으로 추상화한 인프라 클러스터용 운영체제와 같다. Kubernetes는 RESTful 요청을 통해 자원의 상태를 변경하는데, 이때 "정책"이란 이 상태 변경에 대한 Invariant(불변량)를 정의하는 것이다. 상태 변경 요청 시 웹훅을 통해 정책 엔진(Kyverno, OPA Gatekeeper 등)이 이를 인터셉트하여 Invariant를 검사하고, 위반 시 403 오류를 반환하며 요청을 거부한다. 이때 보안 정책과 운영 정책의 구분은 단순히 설계 목적(컨테이너 런타임 권한 상승 방지 vs 분산 시스템 자원 고갈 방지)에 따른 것이다.

대표적인 정책 엔진인 Kyverno는 PolicyException 리소스를 통해 특정 정책 및 rule에 대해 제한적인 예외를 설정할 수 있으나, PaC는 단순 로그 생성이 아닌 실제 배포 차단으로 이어지므로 개발자의 업무 흐름을 중단시키는 문제가 발생한다. 특히, 분산 시스템의 특성상 규칙이 클러스터의 현재 런타임 환경, 컨텍스트 의존성, 복잡한 조건절 연산과 결합하면 개발자가 단일 요청 내에서 위반 원인을 파악하기는 매우 어렵다. 예를 들어, 보안 등급이 높은 네임스페이스(compliance-level=high)에 인프라 도구를 배포할 때 정책이 거부되면, 개발자는 "왜" 거부되었는지에 대한 구체적 설명 없이 단순 거부 응답만을 받게 되며, 이는 불필요한 인지부담을 가중시킨다.

본 프로젝트는 기존 GitOps(Argo CD 등) 환경을 대체하는 것이 아니라, SSoT(Single-Source-of-Truth)로서의 Git 활용을 유지하면서 개발자가 정책 위반 원인과 수정 방향을 쉽게 파악할 수 있도록 돕는 UI 기반의 편의 기능을 제공하는 것을 목표로 한다. 즉, 코드 수정 및 협업 채널 문의 등 분산된 예외 처리 과정을 통합된 그래픽 인터페이스 내로 흡수하여 업무 효율을 높이하고자 한다.

따라서 본 프로젝트의 목적은 Kubernetes 환경에서 PaC 정책 위반이 발생했을 때 개발자가 원인과 수정 방향을 쉽게 파악할 수 있도록 돕고, 임시 정책 예외의 요청·승인·만료·이력 관리를 하나의 시스템에서 수행할 수 있도록 하는 것이다.

구체적인 목표는 다음과 같다.

- 1) 파편화된 정책 위반 정보의 통합 및 가시성 확보
- 2) 통합된 예외 처리 워크플로우 구축
- 3) 예외 생명주기의 자동화 및 이력 보존
- 4) 관리자 의사결정 지원

2. 요구조건 및 제약사항 분석에 대한 수정사항

2.1 요구조건 수정사항

착수보고서 단계에서는 정책 위반 메시지 해석·보강 기능과 정책 예외 처리 기능을 함께 주요 기능으로 정의하였다. 이후 요구사항을 구체화하면서, PaC 위반 대응의 핵심은 단순한 위반 로그 조회가 아니라 배포가 중단된 개발자가 원인·조치 방향·예외 절차를 빠르게 파악하도록 돕는 데 있음을 확인하였다. 이에 따라 중간평가 시점까지는 정책 집행 이후의 대응 과정을 일관되게 관리하는 거버넌스 기능을 우선 구현하기로 하고, 정책 오류 메시지 보강(enrichment) 기능은 예외 처리 기능과 분리하여 후속 구현 항목으로 조정하였다. 이는 예외 요청부터 승인·만료까지의 전체 흐름을 우선 검증하기 위한 결정이다.

수정 및 구체화된 주요 요구조건은 다음과 같다.

구분	착수보고서 계획	중간보고서 구체화	반영 내용
사용자 인증·인가	사용자 역할 구분 필요성 정의	개발자·관리자 권한 분리 및 역할별 기능 접근 제어	JWT 기반 인증 및 RBAC 적용. Access Token은 클라이언트 메모리, Refresh Token은 HttpOnly 쿠키로 관리
정책 위반 조회	위반 정보 통합·가시화	역할에 맞는 정책 위반 현황 조회	위반 정책·대상 리소스·네임스페이스·심각도·처 리 상태·발생 시각 기준 조회 화면/API 설계
예외 요청·승인	예외 요청·승인 기능 정의	제한된 기간의 예외 신청과 근거 기반 승인/거절	요청 사유·대상 정책/클러스터/네임스페이스/리소 스·적용 기간·처리 상태 관리
예외 생명주기	예외 관리 필요성 정의	만료 자동 종료 및 이력 보존	요청·승인·거절·만료 상태와 감사 이력 관리, Kyverno PolicyException 연계
운영 대시보드	대시보드 제공 계획	역할별 요약·목록·필터·상 세 화면	사용자·관리자 대시보드 분리 구성
정책 오류 관리	위반 해석·수정 가이드 병행 구현	후속 단계로 분리	예외 처리 흐름을 우선 구현한 뒤 원인 설명·수정 가이드 기능을 단계적으로 확장

2.2 제약사항 분석 수정사항

시스템 설계와 구현 과정에서 확인된 제약사항을 '제약 → 영향 → 대응 방향'으로 정리하면 다음과 같다.

제약사항	영향	대응 방향
정책 엔진과 서비스 데이터의 정합성	Kyverno 정책·PolicyException 형식이 서비스 데이터 모델과 어긋나면 예외 연계가 실패	정책·예외 데이터 모델을 리소스 형식·적용 범위와 일치시키고, 정책 파일은 소스코드와 분리해 관리. CI에서 정책 린터와 kyverno validate로 기본 유효성 검증
개발 장비 자원 제약	로컬 WSL에서 컨테이너와 kind 클러스터 동시 실행 시 메모리 부족 발생	DevContainer·CLI 자동 설치 스크립트로 환경 표준화, 자원이 필요한 환경은 AWS EC2 원격 개발환경으로 이전
배포 환경의 재현성	개인별 환경 차이로 실행 오류 발생	VSCode DevContainer 자동 구성, CLI 자동 설치 스크립트, 프론트·백엔드 멀티 스테이지 컨테이너 구성
운영 환경과 개발 환경의 차이	kind만으로는 실제 조직 도입 환경을 검증하기 어려움	개발·검증은 kind, 운영 유사 통합 검증은 EKS를 목표로 구성
보안 및 권한 관리	예외 승인이 실제 정책 적용 범위에 영향	관리자 권한을 엄격히 분리하고 모든 상태 변경을 감사 이력으로 보존
단계적 구현 필요성	메시지 보강 동시 구현 시 핵심 흐름 검증 지연	중간평가 시점에는 인증·인가, 위반 가시화, 예외 거버넌스, 인프라 연동을 우선 검증. 테스트는 커밋 단위의 단위 테스트와 전체 흐름의 시스템 테스트로 분리

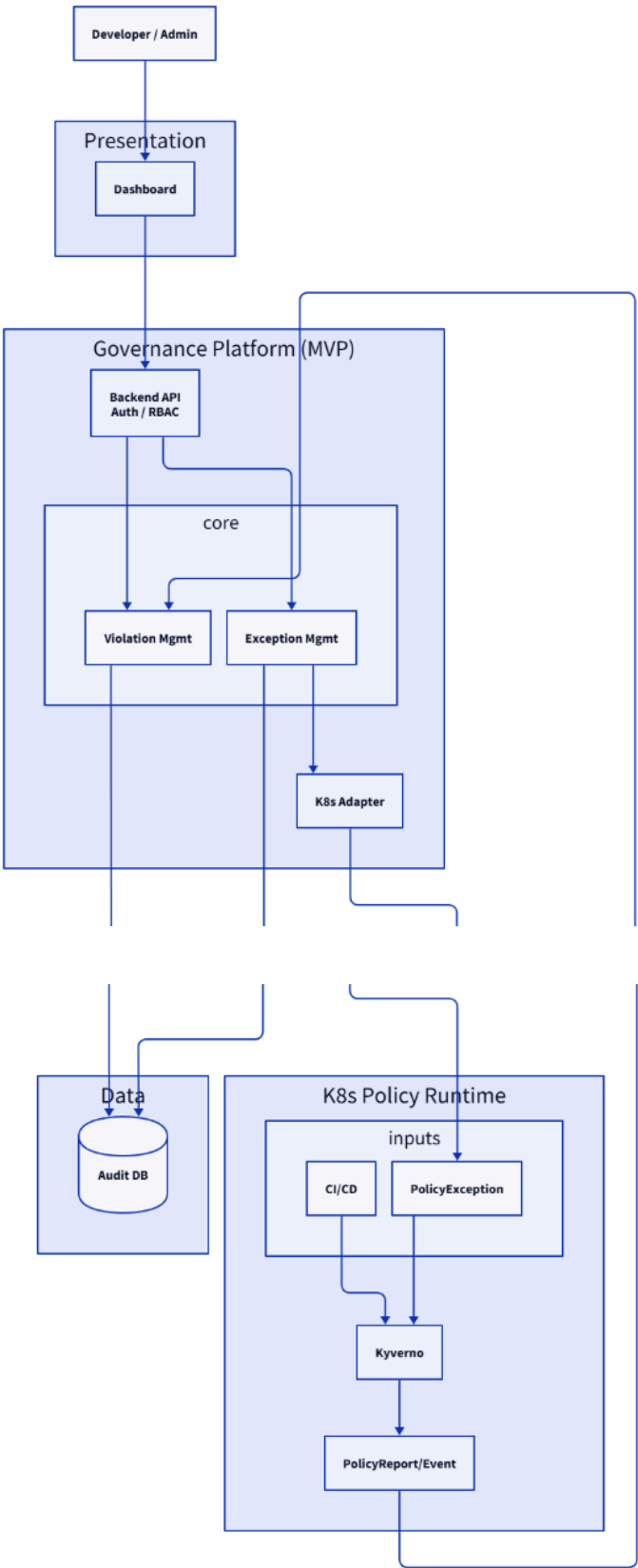
3. 설계 상세화 및 변경 내역

3.1 시스템 구성 상세화

시스템은 프론트엔드, 백엔드 API, 데이터베이스, Kubernetes/Policy Engine, CI/CD 환경으로 구성한다. 사용자 유형은 개발자와 관리자로 구분하며, 공통 인증 체계를 사용하되 접근 가능한 기능과 데이터 범위는 RBAC로 제어한다.

구성 요소	역할
프론트엔드	개발자·관리자별 대시보드, 정책 위반 목록, 예외 요청·승인 화면

	제공. Tailwind CSS·shadcn 기반 공통 UI 사용
인증·인가 API	JWT 기반 로그인·토큰 갱신, Role/Permission 분리 RBAC로 기능 접근 제어, 관리자용 사용자 관리 API 제공
정책 위반 관리 API	정책 엔진에서 발생한 위반 정보를 저장·조회하고 정책·리소스·네임스페이스·심각도·처리 상태 기준으로 관리
예외 처리 API	예외 요청 생성·검토·승인/거절·만료·이력 보존, 승인 예외를 정책 엔진 예외 리소스와 연계
데이터베이스	사용자·역할/권한·클러스터·정책·정책 위반·예외 요청·감사 이력 관리. 스키마 확정 후 물리 구현·더미 데이터 진행 중
Kubernetes/Policy Engine	개발 단계는 kind·Kyverno 검증, 운영 유사 검증은 EKS 배포 계획
CI/CD	소스코드·정책 파일 린트, kyverno validate, 단위/시스템 테스트를 단계적으로 통합



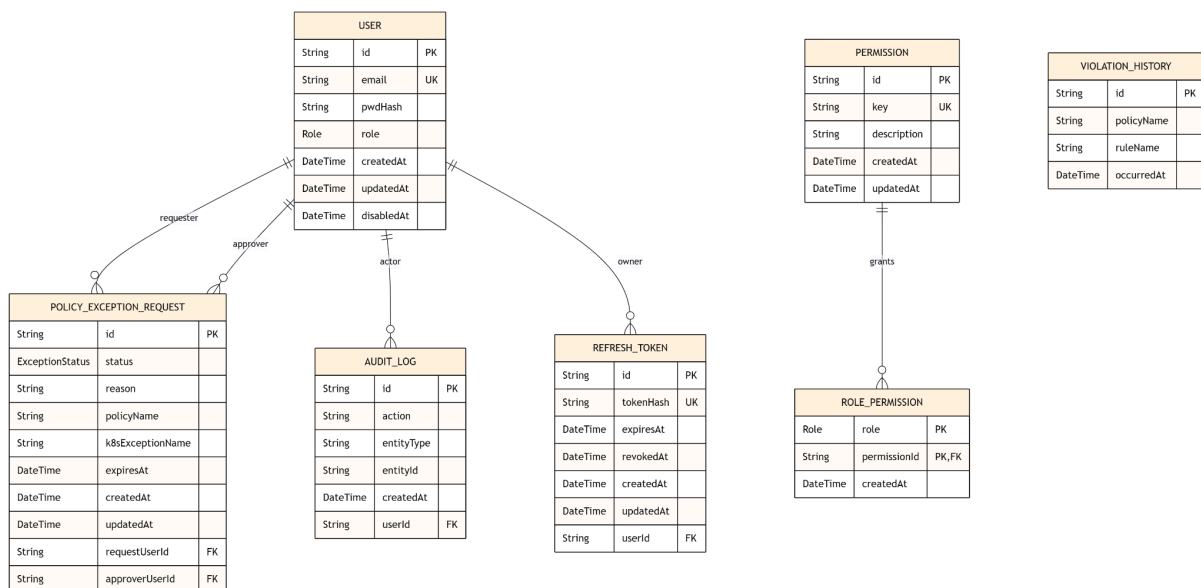
[그림 3-1] 전체 시스템 구성도

3.2 인증·인가 구현 결과

인증 모듈은 다음 API를 구현하였다.

API	기능
POST /auth/login	Access Token과 사용자 정보 반환, Refresh Token을 쿠키에 저장
POST /auth/refresh	Refresh Token 검증·회전 후 새 Access Token 발급
POST /auth/logout	Refresh Token 폐기 및 쿠키 삭제
GET /auth/me	JWT 인증 사용자 정보 조회

인가에는 JWT Guard와 Permission Guard를 적용하였다. ADMIN, APPROVER, REQUESTER, VIEWER 역할과 Permission을 분리하였고, 관리자용 /users API에 users.read, users.create, users.update_role, users.reset_password, users.disable 권한을 적용하였다. 데이터 모델(Prisma)에는 User, Permission, RolePermission, RefreshToken, PolicyExceptionRequest, AuditLog 모델을 정의하여 인증·권한·예외·감사 흐름을 연결하였으며, Refresh Token 처리와 사용자 세션 변경에 대한 단위 테스트·통합 테스트를 추가하였다.



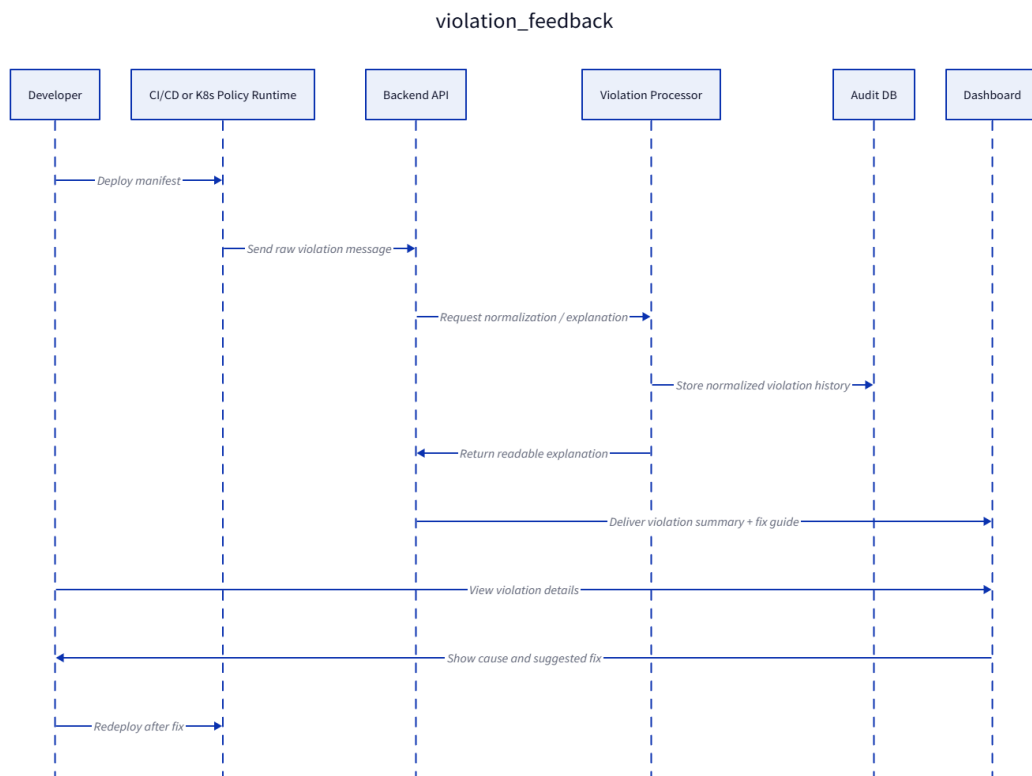
[그림 3-2] Prisma ERD

※ 현재 ERD는 인증·인가와 예외 요청의 1차 모델을 나타내며 클러스터, 정책, 정책 위반 모델은 DB 물리 구현 단계에서 추가

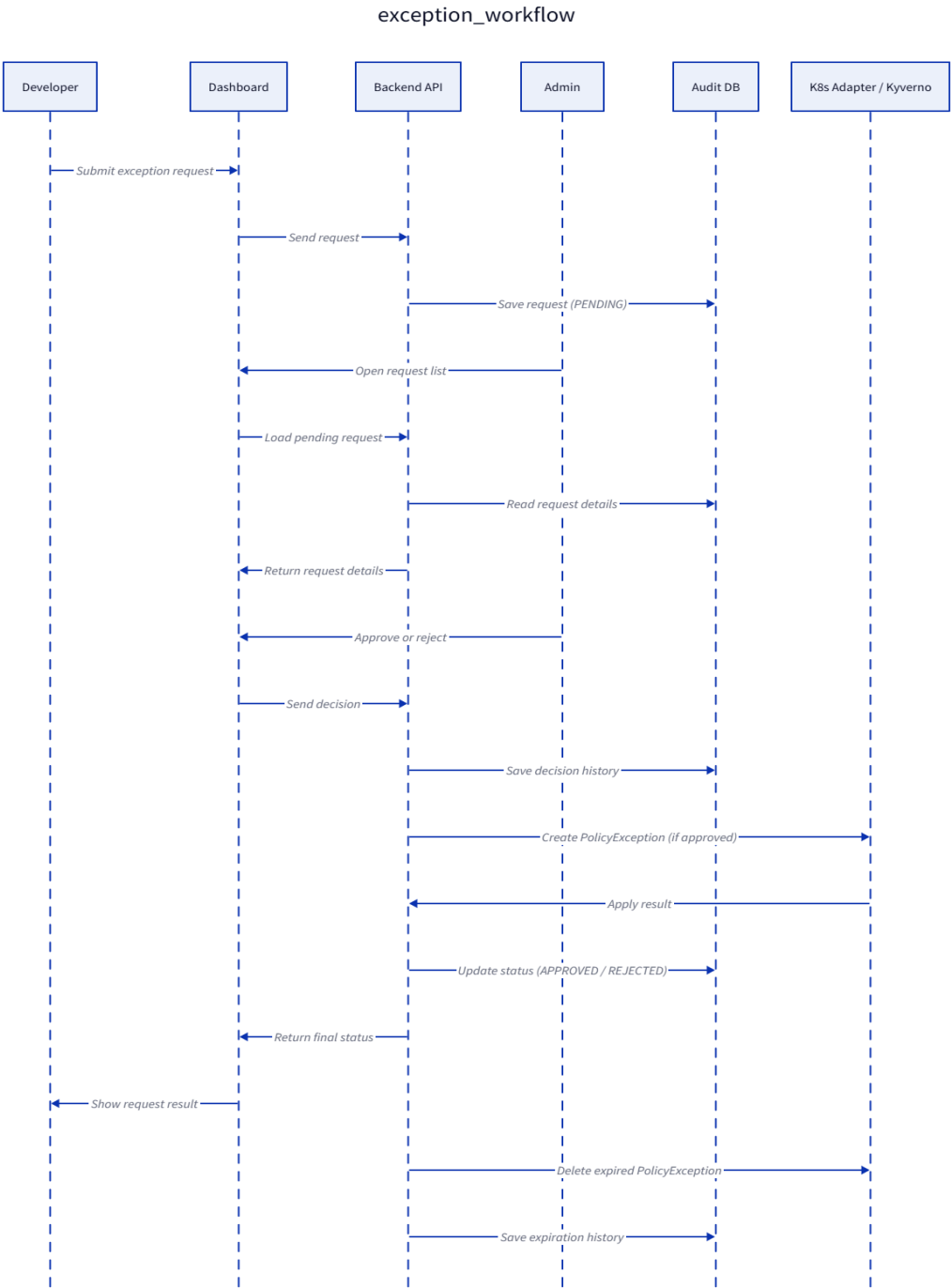
3.3 핵심 업무 흐름

정책 위반 감지 → 위반 정보 조회 → 예외 요청 → 관리자 검토 → 승인/거절 → PolicyException
연계 → 만료 처리 → 감사 이력 보존

- 1) 개발자가 Kubernetes 리소스를 배포·변경하면 Kyverno 정책 검증이 수행된다.
- 2) 정책 위반이 발생하면 위반 정책·대상 리소스·네임스페이스·심각도·발생 시각을 정책 위반 관리 영역에서 조회한다.
- 3) 개발자는 수정이 어려운 상황에서 필요한 범위·기간을 명시하여 예외를 요청한다.
- 4) 관리자는 요청 사유·대상 범위를 검토한 뒤 승인 또는 거절한다.
- 5) 승인된 요청은 제한된 범위의 PolicyException으로 연계하며, 만료 시점에 효력을 종료하고 이력을 보존한다.



[그림 3-3] 정책 위반 피드백 제공 시퀀스 다이어그램



[그림 3-4] 예외 처리 시퀀스 다이어그램

3.4 변경 내역

항목	착수보고서 단계	중간보고서 시점의 상세화·변경	변경 이유
기능 우선순위	정책 위반 해석과 예외 처리 병행 구현	예외 요청·승인·만료·이력 거버넌스 흐름 우선, 메시지 보강은 후속 분리	전체 예외 처리 프로세스를 먼저 구현하고, 인프라 준비가 더 필요한 기능은 추후 구현
정책 검증 환경	Kyverno 중심 활용 계획	kind 기반 개발 검증과 CI의 kyverno validate 연동 우선, EKS 확장 예정	자원 제약·운영 유사성 확보를 단계적으로 분리
인증·인가	역할 구분 필요성 정의	JWT 인증과 Role/Permission 분리 RBAC 구현, 관리자용 사용자 관리 API 구성	보안 영향이 큰 예외 승인의 권한 분리를 조기 확립
UI 설계	대시보드 제공 계획	사용자·관리자 화면 분리, 요약 카드·목록·필터·상세·승인/거절 목록 구현	역할별 정보·조치를 명확히 분리해 검증
테스트 전략	테스트 필요성만 정의	커밋 단위 단위 테스트와 전체 시스템 테스트를 분리해 CI/CD에 통합	회귀 검증과 통합 검증의 목적을 구분

4. 구성원별 진척도

4.1 갱신된 과제 추진 계획

초기 계획상 7월 이전 완료 항목 중 요구사항 분석, 시스템 구조 설계, 사용자 인증·인가 API, 대시보드 디자인은 완료 또는 마무리 단계이며, DB는 논리 스키마를 확정하고 물리 구현을 진행하고 있다. 7월에는 Kubernetes 인프라 및 CI/CD 연동, 정책 엔진 연동, 예외 처리 API, 프론트·백엔드 연동을 집중 수행한다.

(● 완료/마무리, ◐ 진행 중, ○ 예정)

구분	추진 목표	5월	6월	7월	8월	9월
기획	요구사항 분석	●	●			

기획	시스템 구조 설계	●	●	◐		
기획	DB 구현 및 더미 데이터 생성		●	◐		
기획	Kubernetes 인프라 구축 및 CI/CD 파이프라인 연동		◐	◐	○	
기획	테스트케이스 설계			◐	○	
서버 개발	사용자 인증/인가 API		●	●		
서버 개발	Policy Engine API/Kubernetes 연동			◐	○	
서버 개발	정책 예외 처리 API 구현			◐	○	
서버 개발	정책 오류 관리 API 구현				○	○
클라이언트 개발	대시보드 디자인		●	●		
클라이언트 개발	UI/로직 개발			◐	○	
클라이언트 개발	서버 API 연동			◐	○	
클라이언트 개발	UX 점검 및 개선				○	○
공통	테스트 / 최종보고서 작성			◐	○	○

4.2 구성원별 수행 내용

산출물 기준으로 정리하면 다음과 같다.

구성원(담당)	진행 단계	내용
고영림(인프라)	완료	- VSCode DevContainer 자동 구성 - CLI 자동 설치 스크립트 - kind 클러스터 연동 - 프론트·백엔드 멀티 스테이지 컨테이너화 - CI 파이프라인

		- 소스/정책 린터 - PR 템플릿 - kyverno validate 검증 연동 - 시스템 설계 ADR, DB 스키마 확정
	진행 중	DB 인프라 물리 구현
	다음 작업	- EKS 기반 환경 CloudFormation 구성 - 단위/시스템 테스트 파이프라인 통합
최유렬(백엔드)	완료	- JWT 인증(Access Token 메모리, Refresh Token 쿠키) - Refresh Token 회전·폐기 - Role/Permission 분리 RBAC - 관리자용 /users API - 인증·세션 통합 테스트
	진행 중	정책 예외 요청·승인·만료 API 설계
	다음 작업	- 정책 예외 처리 API 구현 - 정책 위반 관리 API 및 오류 관리 API 확장
박태영(프론트엔드)	완료	- Tailwind·shadcn 공통 레이아웃 - 개발자·관리자 대시보드 - 정책·클러스터 현황 - 예외 신청·조회·승인/거절 화면 목업
	진행 중	관리 화면(클러스터·사용자·정책·감사 로그) 보완
	다음 작업	- 백엔드 인증·인가 API 연동 - 예외 처리 API 연동

5. 과제 수행 내용 및 중간 결과

5.1 수행 내용

보고 시점까지 기획·설계, 개발환경 구축, 인증·인가, 사용자 인터페이스의 기반을 우선 확보하였다.

1) 요구사항 및 설계 정리: PaC 위반 대응과 예외 관리 흐름을 중심으로 요구사항을 구체화하고, 인프라·런타임·데이터 모델·테스트 전략에 관한 ADR을 작성하였다.

2) 개발·검증 환경 구축: DevContainer 및 CLI 자동 구성, kind 클러스터 연동, 멀티 스테이지 컨테이너화, CI 파이프라인, 소스코드·정책 린터를 구성하였다. 정책 파일은 CI에서 kyverno validate로 검증되도록 연동하였다.

3) 인증·인가 기능 구현: JWT 기반 인증, Role/Permission 기반 RBAC, 관리자용 사용자 관리 API의 기본 기능을 구현하였다.

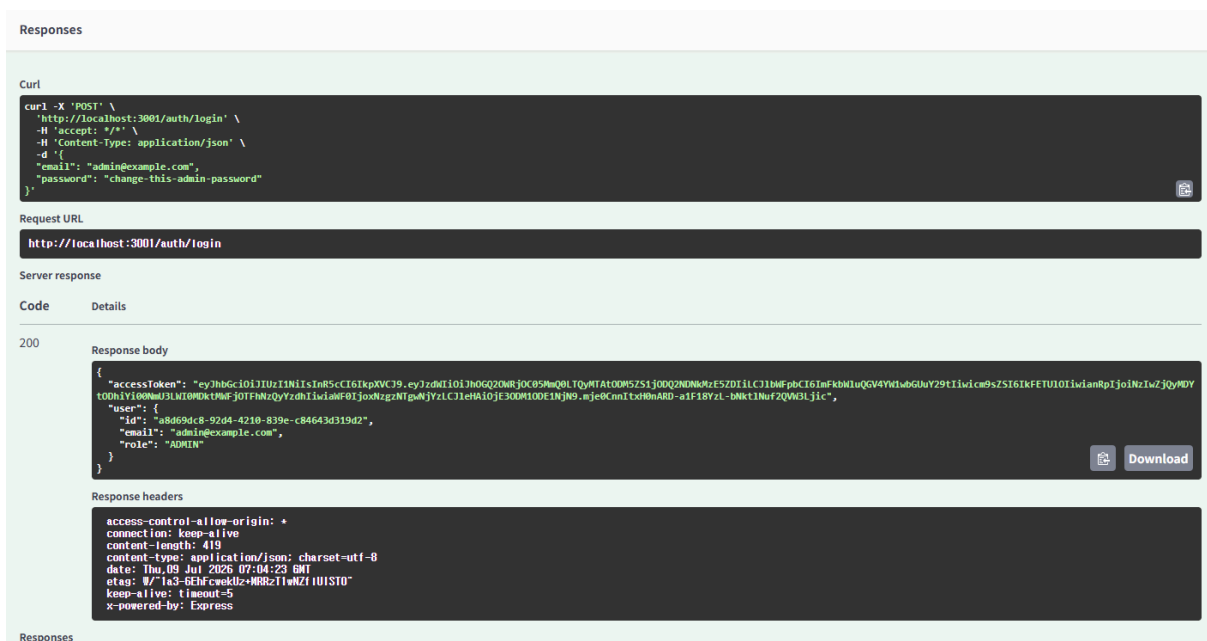
4) 대시보드 및 예외 관리 UI 구현: 개발자·관리자 역할에 맞는 대시보드, 정책 위반 목록, 예외 요청·조회·승인/거절 화면을 목업 데이터 기반으로 구현하였다.

5) 데이터 및 예외 처리 기반 설계: 사용자·권한·정책·클러스터·정책 위반·예외 요청·감사 이력을 관리하는 DB 스키마를 확정하고 예외 처리 API 구현에 착수하였다.

5.2 중간 결과

영역	중간 결과
개발환경	DevContainer·CLI 자동 설치·컨테이너 기반 실행 환경 구성. 자원 제약 환경은 EC2 원격 개발 방식으로 작업 지속 기반 확보
정책 검증	kind·CI 환경에서 kyverno validate 기반 정책 파일 검증 흐름 구성
사용자·권한	개발자·관리자를 구분하는 JWT/RBAC 인증·인가 구조 및 관리자용 사용자 관리 API 구현
사용자 인터페이스	개발자는 정책·클러스터 현황과 예외 요청 상태를, 관리자는 정책 위반 현황과 예외 요청을 확인·처리할 수 있는 화면 구조 확보
통합 기반	예외 요청의 대상·기간·사유·상태와 감사 이력을 관리하는 데이터 모델·업무 흐름 정리로 API·UI·예외 리소스 통합 구현 기반 마련

주요 구현 결과

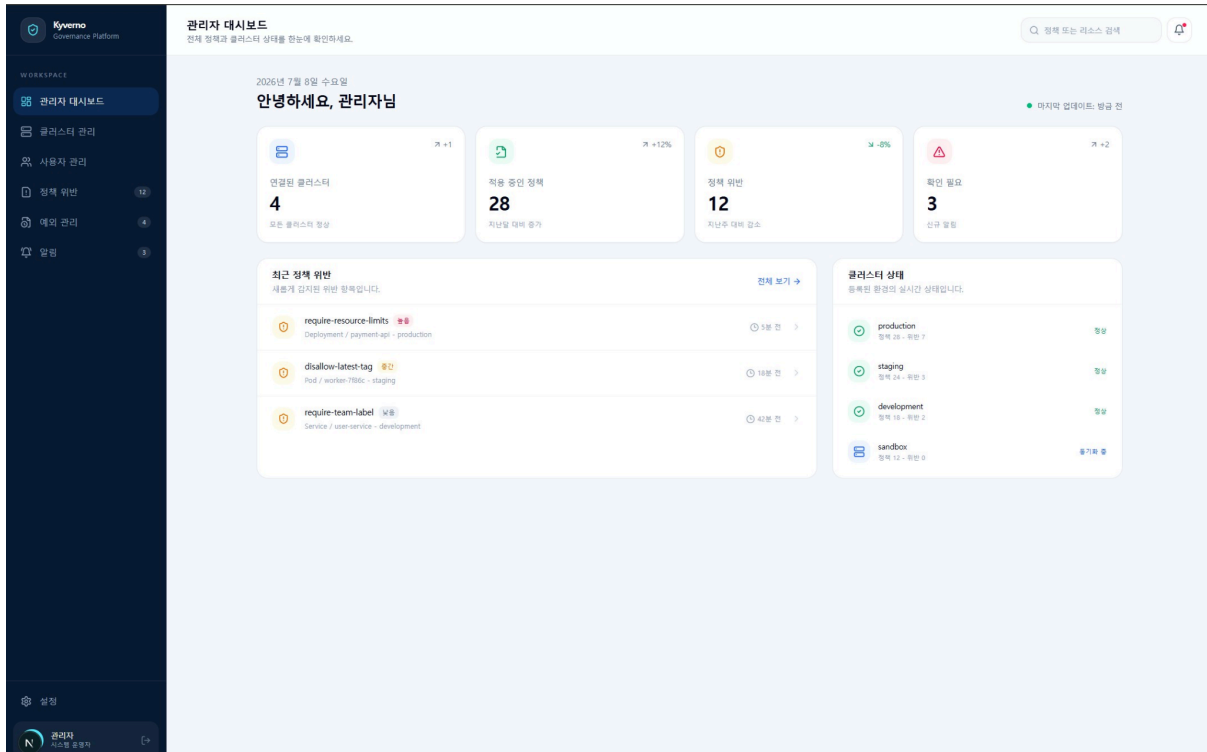


[그림 5-1] 로그인 API Swagger 요청·응답

[그림 5-2] Refresh Token 갱신 요청·응답

[그림 5-3] RBAC 권한 테스트 결과

[그림 5-4] CI에서 kyverno validate 실행 로그



[그림 5-5] 관리자 대시보드 화면

5.3 현재 한계

현재 한계로는 Policy Engine/Kubernetes의 운영 유사 통합과 정책 예외 비즈니스 로직이 완료되지 않았다는 점이 있다. 개발환경상의 정책 검증(kind:CI)은 확보하였으나, 실제 조직 도입과 유사한 EKS 기반 통합 검증과 승인 예외의 PolicyException 연계는 후속 작업으로 남아 있다.

5.4 향후 추진 계획

향후 계획을 작업 단위로 정리하면 다음과 같다.

- 7월 4주차: DB 물리 구현 및 더미 데이터 생성 완료, 정책 예외 요청·승인·거절·만료 관리 API 구현
- 8월 1주차: 예외 처리 API와 프론트엔드 인증·인가/예외 관리 화면 연동
- 8월 2주차: Kubernetes 인프라·CI/CD 연동 마무리, 단위 테스트와 시스템 테스트 분리 통합
- 8월 3주차: EKS 기반 운영 유사 환경에서 Kyverno 정책·예외 리소스 통합 시나리오 검증
- 9월 1주차: 정책, 클러스터, 사용자, 감사 로그 관리 화면 보완 및 UX 점검
- 9월 2주차: 결과 정리, 최종보고서 작성 및 발표자료 준비

본 프로젝트는 현재 기반 인프라와 주요 화면, 인증·인가 구조를 확보한 단계이며, 이후 예외 처리 핵심 로직과 Kubernetes 정책 엔진 연동을 완료하여 PaC 위반 대응 및 예외 관리의 전체 흐름을 검증할 예정이다.